Functions in C++

(User defined functions)

What is a function?

A function in C++ language is a block of code that performs a specific task. It has a name and it is reusable i.e. it can be executed from as many different parts in a C++ Program as required. It also optionally returns a value to the calling program

Properties of C++ Functions

- Every function has a unique name. This name is used to call function from “main()” function. A function can be called from within another function.

- A function is independent and it can perform its task without intervention from or interfering with other parts of the program.

Properties of C++ Functions(cont…)

- A function performs a specific task. A task is a distinct job that your program must perform as a part of its overall operation, such as adding two or more integer, sorting an array into numerical order, or calculating a cube root etc.

Properties of C Functions(cont…)

- A function returns a value to the calling program. This is optional and depends upon the task your function is going to accomplish. Suppose you want to just show few lines through function then it is not necessary to return a value. But if you are calculating area of rectangle and wanted to use result somewhere in program then you have to send back (return) value to the calling function.

Types of functions in C++

- void function(void)
- void function(int)
- int function(void)
- int function(int)

void function(void)

Type-1

Does not return a value

```
void function(void)
```

Takes no arguments

```
#include<conio.h>
#include<stdio.h>

void sample_func(void);

void main()
    {
    clrscr();

    sample_func();

    getch();
    }

void sample_func(void)
    {
    printf("ISE-RVCE");
    }
```

Output

ISE-RVCE

We can call a function for more than once in main.

```
#include<conio.h>
#include<stdio.h>

void sample_func(void);

void main()
    {
    clrscr();

     sample_func();
     sample_func();
    getch();
    }

void sample_func(void)
    {
    printf("ISE-RVCE ");
    }
```

Output

ISE-RVCE ISE-RVCE

We can build/call more than one functions in our program.

```
#include<conio.h>
#include<stdio.h>

void sample_func(void);
void sample_func1(void);

void main()
    {
    clrscr();
     sample_func();
     sample_func1();
    getch();
    }

void sample_func(void)
    {
    printf("CSE-RVCE\n");
    }

void sample_func1 (void)
    {
    printf("ISE-RVCE");
    }
```

Output

CSE-RVCE

ISE-RVCE

Type-2

void function(int)

Type-2

Does not return a value

```
void function(int)
```

But takes Argument(s) of type int

```
#include<conio.h>
#include<stdio.h>

void square(int);

void main()
    {
    clrscr();

    square(5);

    getch();
    }

void square(int x)
    {
    printf("Square is %d",x*x);
    }
```

```
#include<conio.h>
#include<stdio.h>

void square(int);

void main()
    {
    clrscr();

    square(5);

    getch();
    }

void square(int x)
    {
    printf("Square is %d",x*x);
    }
```

Output

Square is 25

```
#include<conio.h>
#include<stdio.h>

void table(int);

void main()
    {
    clrscr();

        table(5);

    getch();
    }

void table(int x)
    {
    for(int a=1;a<=10;a++)
    printf("%d*%d=%d\n",a,x,a*x);
    }
```

```
#include<conio.h>
#include<stdio.h>

void table(int);

void main()
    {
    clrscr();

       table(5);

    getch();
    }

void table(int x)
    {
    for(int a=1;a<=10;a++)
    printf("%d*%d=%d\n",a,x,a*x);
    }
```

Output

1*5=5
2*5=10
3*5=15
4*5=20
5*5=25
6*5=30
7*5=35
8*5=40
9*5=45
10*5=50

We can pass more than one arguments to a function, each can have different type/same type.

```
#include<conio.h>
#include<stdio.h>

void add(int, int);

void main()
    {
    clrscr();

      add(5, 7);

    getch();
    }

void add(int x,    int y)
    {
    printf(“Addition is %d",x+y);
    }
```

```
#include<conio.h>
#include<stdio.h>

void add(int, int);

void main()
    {
    clrscr();

        add(5, 7);

    getch();
    }

void add(int x,    int y)
    {
    printf("Addition is %d",x+y);
    }
```

Output

Addition is 12

Write a program to generate a loop between two numbers.(Use functions to achieve the task)

```
#include<conio.h>
#include<stdio.h>

Void  makeloop(int, int);

void main()
    {
    clrscr();

      makeloop (2, 15);

    getch();
    }

void makeloop (int x,    int y)
    {
for(int a=x;   a<=y;   a++)
    printf("%d,", a);
    }
```

```
#include<conio.h>
#include<stdio.h>

Void  makeloop(int, int);

void main()
    {
    clrscr();

            makeloop (2, 15);

    getch();
    }

void makeloop (int x,    int y)
    {
 for(int a=x;   a<=y;   a++)
    printf("%d,", a);
    }
```

Output

2,3,4,5,6,7,8,9,10,11,12,13,14,15

Edit same program, get two numbers input from user, then make a loop between these numbers.

Ex:

Plzz input two numbers: 5 20

5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20

```
#include<conio.h>
#include<stdio.h>

Void  makeloop(int, int);

void main()
     {
     clrscr();
     int start, end

     printf("Plzz input two numbers:");
     scanf("%d%d", start, end);

       makeloop (start, end);

     getch();
     }

void makeloop (int x,    int y)
     {
for(int a=x;   a<=y;   a++)
     printf("%d,", a);
     }
```

Type-3

int function(void)

Type-3

Returns a value of int type.

```
int function(void)
```

But takes no arguments.

```
#include<conio.h>
#include<stdio.h>

int  sum(void);

void main()
      {
      clrscr();
      int a;

      a=sum();
        printf("Sum is %d", a);
        getch();

      }

int sum(void)
      {
      int x=2, y=3, z;
      z=x+y;

      return(z);
      }
```

```
#include<conio.h>
#include<stdio.h>

int  sum(void);

void main()
      {
      clrscr();
      int a;

      a=sum();
        printf("Sum is %d", a);
        getch();

      }

int sum(void)
      {
      int x=2, y=3, z;
      z=x+y;

      return(z);
      }
```

Return Keyword

- In functions, where we return any value, we usually use a keyword “return”, and the value, which is to be returned is written inside braces, shown after return.
- Example: return(z);
- The above statement will pass the value of z to the left side variable of calling function.
- ie: a=sum();
- Hence the value in z, which is 5 in our program is passed to a.

Type-4

Returns a value.

int function(int)

Passes the argument(s).

```
#include<conio.h>
#include<stdio.h>

int  sum(void);

void main()
      {
      clrscr();
      int a;

      a=sum(2,3);
        printf("Sum is %d", a);
        getch();

      }

int sum(int x, int y)
      {

      z=x+y;

      return(z);
      }
```

```
#include<conio.h>
#include<stdio.h>

int  sum(int, int);

void main()
      {
      clrscr();
      int a;

      a=sum(2,3);
        printf("Sum is %d", a);
        getch();

      }

int sum(int x, int y)
      {

      z=x+y;

      return(z);
      }
```